Designing an AI-Based System to Solve a Real-World Problem: Spam vs Legitimate Message Classifier

Course: Software Design in Intelligent Systems
Student: Maftuna Sharabbayeva
Student ID: 380261200262
University: Muhammad al-Khwarizmi Tashkent University of Information Technologies
Selected domain: Cybersecurity: classify messages as spam or legitimate

1. Problem Definition and Importance

Spam messages (SMS, email, messenger) flood users with unwanted advertising, and a large share of them are phishing or fraud attempts. They try to steal passwords, bank card numbers and money by imitating banks, delivery services or prize promotions. Manually checking every message is slow, and users who miss a malicious message can suffer financial loss and identity theft.

Target users: ordinary mobile and email users, university staff and students, and administrators of mail or messaging services who want an automatic first-line filter.

Expected benefit of an AI-based solution: fixed keyword blacklists are easy to bypass, because spammers change wording constantly. A machine-learning classifier learns word patterns from labelled examples and can be retrained as new spam appears. It filters messages instantly and consistently, and the final decision on suspicious messages can still be left to the user.

2. System Requirements

Functional requirements

1. Accept a text message as input.
2. Pre-process the text (lower-casing, URL and number normalisation, punctuation removal).
3. Classify the message as **spam** or **legitimate**.
4. Output a confidence value (probability) with the label.
5. Report evaluation metrics (accuracy, precision, recall, F1, confusion matrix).

6. Allow retraining when new labelled data is added.

Input data: plain-text messages.
Output: label (spam / legitimate) plus a probability.

Constraints: small prototype dataset; text only (no images or attachments); English-language messages; must run on a normal laptop with Python and scikit-learn. Output is advisory: borderline messages should be reviewed by a human.

3. System Architecture

The system is a linear pipeline with five components:

1. **Data layer:** labelled messages stored in a CSV file (`messages_dataset.csv`).
2. **Pre-processing module:** lower-casing, replacing links with `urltoken`, replacing money and long numbers with `numtoken`, removing punctuation and stop words.
3. **Feature extraction:** TF-IDF vectoriser using unigrams and bigrams, which turns each message into a numeric vector.
4. **Classifier:** Multinomial Naive Bayes, trained on 75% of the data.
5. **Interface / output module:** `classify()` function takes a new message and returns label and probability; an evaluation module prints the metrics.

Data flow:

`User message` → `Pre-processing` → `TF-IDF vector` → `Naive Bayes classifier` → `Label + confidence` → `User / human review`

Training flow: `Dataset` → `train/test split (75/25)` → `Pre-processing + TF-IDF` → `fit Naive Bayes` → `evaluate on test set`

4. Data Preparation

Source: a synthetic labelled dataset of 200 messages (100 spam, 100 legitimate) generated by the script from hand-written message templates. Each template has slots (names, links, prizes, amounts, times, course names) filled randomly, so messages vary. The file is saved as `messages_dataset.csv`.

Features: the words and word pairs of the message text, weighted by TF-IDF.

Labels: `spam` or `legitimate`.

- Spam templates imitate prize scams, phishing ("verify your password"), fake loans, work-from-home offers and fake parcel fees.
- Legitimate templates imitate everyday student messages: meeting arrangements, lecture notes, deadlines, family messages.

Pre-processing steps: lower-casing; URLs replaced by `urltoken`; amounts like `$5000` replaced by `numtoken`; punctuation removed; English stop words removed; whitespace normalised.

Split: 75% training (150 messages) and 25% testing (50 messages), stratified so both classes are balanced.

Note: a larger public dataset such as the UCI SMS Spam Collection can replace the synthetic file without changing the code.

5. Selected AI Method and Justification

Method: TF-IDF features with a **Multinomial Naive Bayes** classifier (supervised machine learning).

Why it is appropriate:

- Naive Bayes is the classic, well-proven algorithm for text and spam filtering.
- It trains very quickly and works well on small datasets.
- It outputs probabilities, which are useful for flagging borderline messages for human review.
- It is simple to explain: it learns which words appear more often in spam than in legitimate messages.
- TF-IDF reduces the weight of very common words and highlights distinctive ones such as "winner", "urgent" and "verify".

Alternatives considered: logistic regression and linear SVM (often slightly more accurate on large data, but less naturally probabilistic and slower to explain); a pure keyword rule-based filter (easy to evade); deep learning models such as transformers (more powerful, but unnecessary and too heavy for this prototype).

6. Prototype Implementation

The full source code is in `spam_classifier.py` (language: Python; libraries: pandas, scikit-learn). Core parts:

```
def preprocess(text: str) -> str:
    text = text.lower()
    text = re.sub(r"http\S+|www\.\S+", " urltoken ", text)
    text = re.sub(r"\$\d+|\d{3,}", " numtoken ", text)
    text = re.sub(r"[^a-z\s]", " ", text)
    return re.sub(r"\s+", " ", text).strip()

def make_model():
    return Pipeline([
        ("tfidf", TfidfVectorizer(preprocessor=preprocess,
                                  stop_words="english", ngram_range=(1, 2))),
        ("nb", MultinomialNB(alpha=0.5)),
    ])

def classify(model, message):
    proba = model.predict_proba([message])[0]
    label = list(model.classes_)[proba.argmax()]
    return label, round(float(proba.max()), 3)
```

Run with `python spam_classifier.py`. The script builds the dataset, trains the model, prints the evaluation metrics, and classifies eight new test messages.

7. Testing

Eight new, hand-written messages (not produced by the training templates) were classified by the final model. Actual output of the program:

#	Input message	Expected	Output	Confidence
1	Congratulations! You won a free iPhone. Click http://claim-prize.top now	Spam	**Spam**	0.991
2	Hi Madina, can we move the AI Systems lab to 3 pm tomorrow?	Legitimate	**Legitimate**	0.990
3	URGENT: verify your bank password at www.secure-bank-login.biz or account will be closed	Spam	**Spam**	0.997
4	Please send me the slides from today's Databases lecture, thanks	Legitimate	**Legitimate**	0.935

#	Input message	Expected	Output	Confidence
5	Earn $3000 per week from home, no experience needed!!!	Spam	**Spam**	0.973
6	Mom says dinner is ready, come home soon	Legitimate	**Legitimate**	0.850
7	Your invoice is attached. Let me know if the payment details are correct	Legitimate	**Legitimate**	0.933
8	Limited offer: 80% discount on luxury watches. Buy now!	Spam	**Spam**	0.987

All 8 test cases were classified as expected. The lowest confidence (0.85) occurred for the short family message, which has few informative words, so lower confidence there is reasonable.

8. Evaluation of Results

Evaluation on the 50-message hold-out test set (spam is the positive class):

Metric	Value
Accuracy	1.00
Precision	1.00
Recall	1.00
F1-score	1.00

Confusion matrix (rows = true class, columns = predicted class):

	Predicted legitimate	Predicted spam
Actual legitimate	25	0
Actual spam	0	25

Interpretation. The perfect scores should **not** be read as proof that the system would be perfect in real life. The dataset is synthetic and built from a small set of templates, so the test messages are very similar to the training messages, which makes the task easy. The

eight hand-written test cases are a more realistic check, and they were all correct, but eight examples is still a very small sample. In a real filter, precision matters most because a false positive (a legitimate message marked as spam) can hide an important message, while recall matters because a missed spam may be a phishing attack.

9. Limitations

- The dataset is small and synthetic, with limited vocabulary and few writing styles.
- Only English text is supported; Uzbek and Russian messages (very common locally) would need new training data.
- The model uses bag-of-words and ignores word order and deeper meaning, so cleverly written phishing messages or messages with deliberate misspellings ("fr33 pr1ze") may pass.
- Short messages carry little information and give low-confidence results.
- Spam changes over time, so the model will become outdated without retraining.
- It does not examine sender reputation, links' real destinations, or attachments.
- Output must support, not replace, human judgment.

10. Proposed Improvements

1. Train on a large public dataset (e.g. UCI SMS Spam Collection) with thousands of real messages and use cross-validation.
2. Add Uzbek and Russian messages and multilingual preprocessing.
3. Add extra features: number of links, share of capital letters, exclamation marks, sender information.
4. Compare with logistic regression, linear SVM and a transformer-based model.
5. Add a confidence threshold: messages in an uncertain range go to a "review" folder instead of being auto-deleted.
6. Build a simple Streamlit interface and a feedback button so users can report misclassified messages for retraining.

11. Conclusion

This laboratory work designed and implemented a working AI prototype for a cybersecurity problem: classifying messages as spam or legitimate. A TF-IDF plus Naive Bayes pipeline was built, tested on eight new messages (all correct) and evaluated on a hold-out set. The prototype shows the full development cycle: problem definition, requirements, architecture, data preparation, method selection, implementation, testing and critical evaluation. Its limitations, mainly the small synthetic dataset, show that further work with real data is needed before practical use.

12. Answers to Self-Assessment Questions

1. **Real-world problem and beneficiaries:** spam and phishing messages; email and SMS users, students, staff and service administrators benefit.
2. **Main components:** dataset, pre-processing module, TF-IDF feature extractor, Naive Bayes classifier, and the prediction/evaluation interface.
3. **AI method and why:** TF-IDF with Multinomial Naive Bayes, because it is fast, accurate on text, works with small data, gives probabilities and is easy to explain.
4. **Data and preparation:** 200 synthetic labelled messages (100 spam, 100 legitimate); lower-casing, URL/number normalisation, punctuation and stop-word removal, then a 75/25 split.
5. **Testing and evaluation:** eight new test messages plus accuracy, precision, recall, F1 and a confusion matrix on a hold-out set.
6. **Limitations and improvements:** small synthetic data, English only, no word order or sender analysis; improve with real datasets, multilingual data, more features, stronger models and a feedback loop.